PiCar

1. Beschreibung

Das PiCar-X ist ein kleines Roboterauto für Lernzwecke. Die Firma SunFounder baut es. Ein **Raspberry Pi** (ein kleiner Computer) steuert das Auto. Man kann es mit **Python** programmieren oder mit Blockly (Programmieren mit Blöcken).

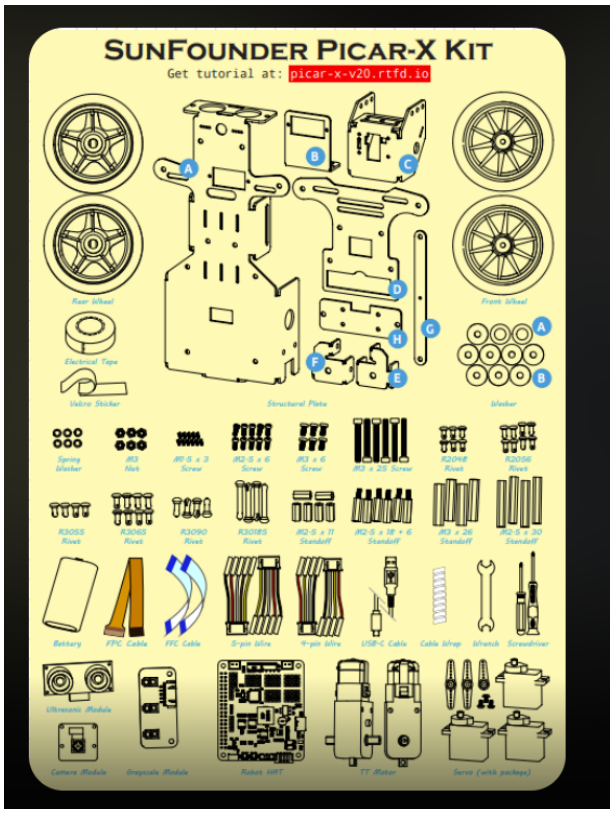
2. Funktionen

- Fahren und lenken (vorwärts, rückwärts, links, rechts).
- Mit der Kamera ein Live-Bild zeigen. Die Kamera kann sich drehen und neigen.
- Hindernisse erkennen (Ultraschall-Sensor) und automatisch ausweichen.
- Einer Linie auf dem Boden folgen (Graustufen-Sensor).
- Abgründe erkennen, damit das Auto nicht herunterfällt.
- Farben, Gesichter und Objekte erkennen (Computer Vision).
- Sprechen (Text-to-Speech) und Töne abspielen (Lautsprecher).
- Per Handy-App mit Live-Bild oder per Tastatur steuern.

3. Aufbau und Software

Hardware

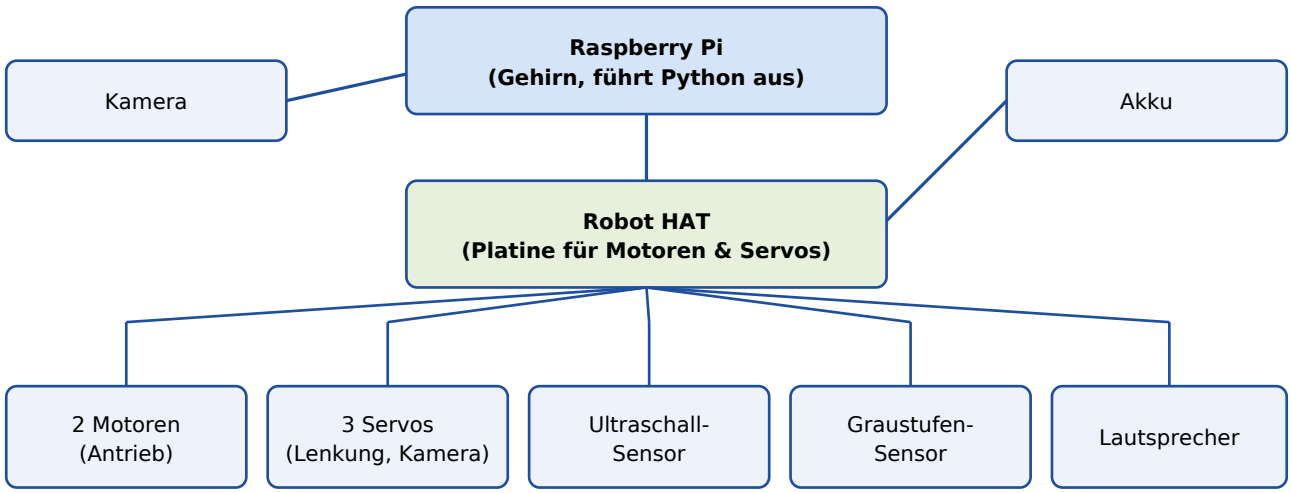
Der Raspberry Pi gehört nicht zum Bausatz.





Teil	Aufgabe
Raspberry Pi	Der Computer des Autos. Er führt das Programm aus.
Robot HAT	Die Platine auf dem Pi. Sie steuert Motoren und Servos und hat einen Lautsprecher.
2 Motoren	Sie treiben die Räder an.
3 Servos	Einer lenkt die Vorderräder. Zwei bewegen die Kamera (drehen und neigen).
Kamera	Sie liefert das Live-Bild.
Ultraschall-Sensor	Er misst den Abstand zu Hindernissen.
Graustufen-Sensor	Er erkennt Linien auf dem Boden und Abgründe.
Akku	Er gibt dem Auto Strom. Ohne Akku laufen die Motoren nicht.

So hängen die Teile zusammen



Software

- **Raspberry Pi OS:** das Betriebssystem (Linux) auf der SD-Karte.
- **Python:** die Programmiersprache.
- **robot-hat:** Bibliothek für Motoren und Servos.
- **vilib:** Bibliothek für die Kamera.
- **picar-x:** Bibliothek mit der Klasse für das Auto.

4. Einrichtung

Der Raspberry Pi gehört nicht zum Bausatz. Außerdem wird eine microSD-Karte benötigt.

Vorbereitung

- Raspberry Pi OS wird mit dem **Raspberry Pi Imager** auf die SD-Karte geschrieben. Dabei werden WLAN und SSH eingestellt.
- Der PC wird per **SSH** mit dem Raspberry Pi verbunden.
- **I2C** wird aktiviert: `sudo raspi-config` → Interface Options → I2C → Enable. Über I2C spricht der Pi mit dem Robot HAT.

Die Befehle

Alle Befehle gibt man im Raspberry Pi ein (per SSH). Der Pi braucht Internet.

a) System aktualisieren

```
sudo apt update
sudo apt upgrade
```

- `sudo`: führt den Befehl als Administrator aus.
- `apt update`: lädt die Liste der verfügbaren Pakete.
- `apt upgrade`: installiert neue Versionen der vorhandenen Programme.

b) Nur bei Raspberry Pi OS Lite: Grundprogramme installieren

```
sudo apt install git python3-pip python3-setuptools python3-smbus
```

Das installiert `git` (Dateien von GitHub laden), `pip` (Python-Pakete installieren), `setuptools` und `smbus` (Kommunikation über I2C).

c) robot-hat installieren (Motoren und Servos)

```
cd ~/
git clone -b 2.5.x https://github.com/sunfounder/robot-hat.git --depth 1
cd robot-hat
sudo python3 install.py
```

- `cd ~/`: wechselt in den Home-Ordner.
- `git clone -b 2.5.x ... --depth 1`: lädt Version 2.5.x des Projekts von GitHub. `--depth 1` lädt nur die neueste Version. Das geht schneller.
- `cd robot-hat`: wechselt in den geladenen Ordner.
- `sudo python3 install.py`: startet das Installations-Skript.

d) vilib installieren (Kamera)

```
cd ~/
git clone https://github.com/sunfounder/vilib.git --depth 1
cd vilib
sudo python3 install.py
```

Gleiche Schritte wie bei robot-hat.

e) picar-x installieren (das Auto)

```
cd ~/
git clone -b 2.1.x https://github.com/sunfounder/picar-x.git --depth 1
cd picar-x
sudo pip3 install .
```

Hier installiert `pip3 install .` das Paket aus dem aktuellen Ordner (der Punkt). Bei dem Fehler `externally-managed-environment` wird `--break-system-packages` an den Befehl angehängt.

f) Ton aktivieren (optional)

```
cd ~/robot-hat
sudo bash i2samp.sh
```

Das Skript richtet den Lautsprecher ein. Die Fragen werden mit y beantwortet. Danach wird das System neu gestartet.

5. Inbetriebnahme

Schritt 1: Akku

Der Akku muss geladen und angeschlossen sein, und der Schalter am Robot HAT muss eingeschaltet sein. Nur mit USB-Strom läuft der Pi, aber die Motoren laufen nicht.

Schritt 2: Kalibrieren

Beim Bau entstehen kleine Abweichungen. Ohne Kalibrierung fährt das Auto eventuell schief. Dabei sollten die Räder in der Luft sein.

```
cd ~/picar-x/example
sudo python3 1.cali_servo_motor.py
```

- **R**: testet, ob die 3 Servos funktionieren.
- **1, 2 oder 3**: wählt den Lenk-Servo, den Servo für Kamera links/rechts oder den Servo für Kamera hoch/runter. Mit **W** und **S** wird er gerade gestellt.
- **E**: testet, ob das Auto vorwärts fährt. Wenn ein Motor falsch herum dreht, wird er mit **4** oder **5** gewählt und mit **Q** umgekehrt.
- **Leertaste**: speichert die Werte. Danach wird mit y bestätigt und mit **Strg+C** beendet.

Es gibt auch eine Kalibrierung für den Graustufen-Sensor. Sie ist nur für die Linienverfolgung nötig (`1.cali_grayscale.py`).

Schritt 3: Beispiel starten

```
cd ~/picar-x/example
sudo python3 1.move.py
```

Ein Forum von SunFounder nennt `1.move.py` als Beispiel für die manuelle Steuerung. Die benutzten Tasten werden im Terminal angezeigt. **Dafür ist keine Programmierung nötig:** Die Beispiele sind schon fertig.

Schritt 4 (optional): Eigenes Programm

Mit der Bibliothek `picarx` kann man das Auto selbst steuern. Ein kurzes Beispiel:

```
from picarx import Picarx
import time

px = Picarx()

px.forward(30)                # fährt vorwärts, Geschwindigkeit 30
time.sleep(2)                 # wartet 2 Sekunden (das Auto fährt weiter)

px.set_dir_servo_angle(20)    # lenkt 20 Grad nach rechts
time.sleep(1)

px.set_dir_servo_angle(0)     # Räder wieder gerade
px.stop()                     # hält das Auto an
```

- `Picarx()` erstellt das Auto-Objekt.
- `forward(30)` startet die Motoren. Die Zahl ist die Geschwindigkeit.
- `time.sleep(2)` pausiert das Programm 2 Sekunden. Die Motoren behalten ihren letzten Befehl, das Auto fährt also weiter.
- `set_dir_servo_angle()` bewegt den Lenk-Servo.
- `stop()` hält die Motoren an. Ein Programm sollte immer mit `stop()` enden.

Hinweis: Die Methodennamen wurden nicht in der Dokumentation überprüft. Sie sollten vor dem Start des Programms mit der Dokumentation oder den Beispielen im Ordner `~/picar-x/example` verglichen werden.

Schritt 5: Steuerung per Handy mit Live-Bild

Die App **SunFounder Controller** (iOS und Android) hat Joystick, Tasten und ein Live-Bild von der Kamera.

```
cd ~
git clone https://github.com/sunfounder/sunfounder-controller.git
cd ~/sunfounder-controller
sudo python3 setup.py install

cd ~/picar-x/example
sudo python3 13.app_control.py
```

- Die App wird installiert, und mit **+** wird ein neuer Controller erstellt. Dort wird **PiCar-X** gewählt.
- Das Handy muss im **gleichen WLAN** sein wie das PiCar-X.
- Mit **Run** wird gestartet: Das Kamerabild wird angezeigt, und das Auto kann gesteuert werden.

Quelle

Offizielle Dokumentation von SunFounder: https://docs.sunfounder.com/projects/picar-x-v20/en/latest/